

SmartRetail POS Intelligence Pipeline

Project Overview & Documentation

What Is This Project?

The SmartRetail POS Intelligence Pipeline is a data engineering project that simulates and processes point-of-sale (POS) data for a retail chain. It takes raw, messy transaction data and transforms it into clean, reliable analytical tables that a business could use to make decisions about sales performance, inventory levels, customer behaviour, and promotions.

The entire pipeline runs on free tools with no monthly cost.

The Business Problem It Solves

Retail businesses generate enormous amounts of transaction data every day across hundreds of stores. The challenge is that this raw data is never clean — it contains duplicate records, missing values, invalid store IDs, future-dated transactions, and negative quantities. Before any analysis can happen, this data needs to be cleaned, validated, and structured.

On top of that, business teams need answers to questions like:

- Which stores are generating the most revenue?
- Which products are at risk of running out of stock?
- Who are our most valuable customers?
- Are our promotions actually driving sales?
- How much revenue are we losing to discounts?

This pipeline answers all of those questions automatically, every day, with data quality checks built in to ensure the numbers can be trusted.

The Data

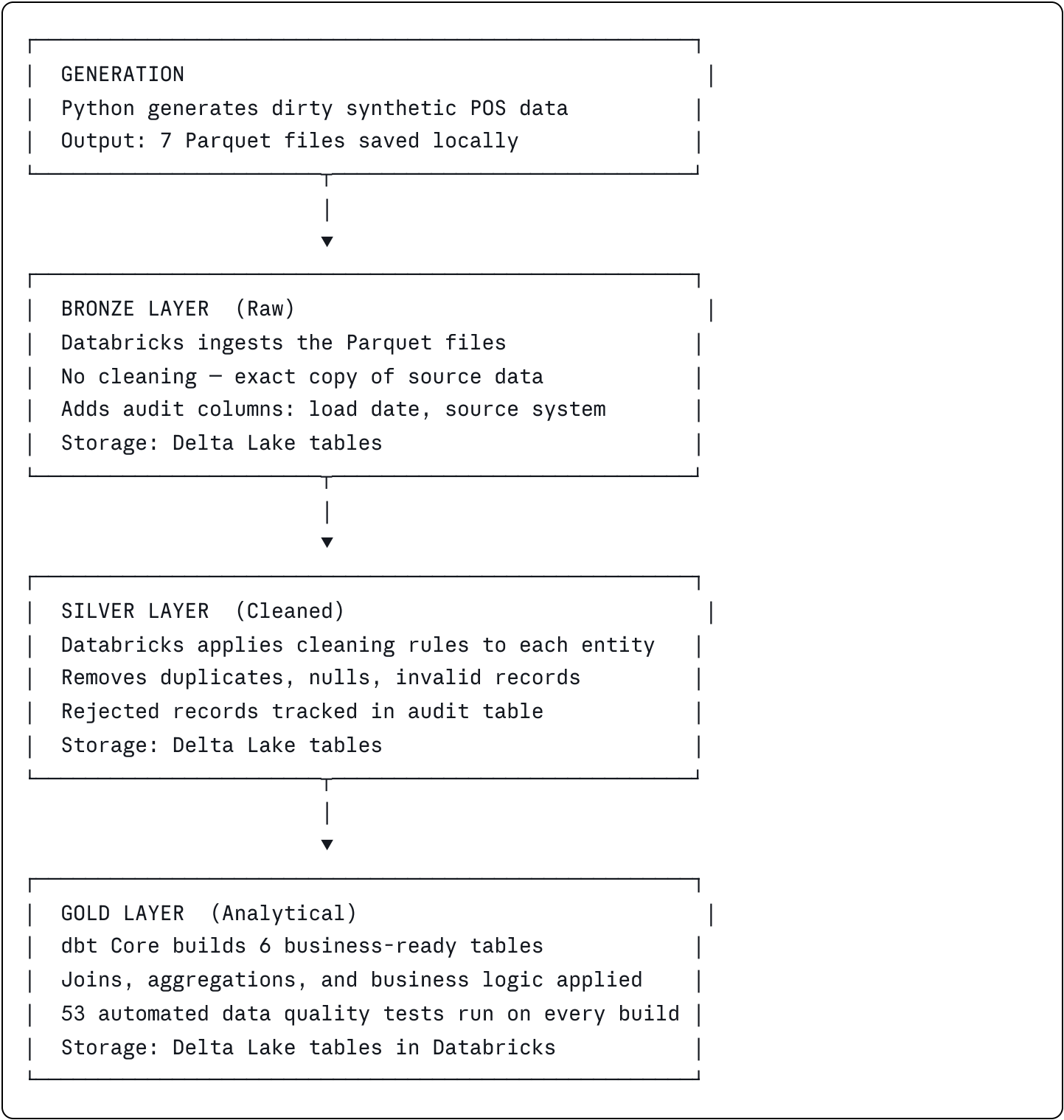
The pipeline generates synthetic (fake but realistic) retail data using Python. Each daily run produces:

Entity	Volume	What it represents
Stores	20 records	Physical and online store locations across 4 regions
Products	500 records	Items across 5 categories with cost and price data
Customers	10,000 records	Registered loyalty programme members
Transaction Headers	~5,000 per day	One record per sale (store, customer, time, payment)
Transaction Lines	~22,000 per day	One record per item within each sale
Inventory	1,000 records	Stock levels per store/product combination
Promotions	50 records	Active discount campaigns

Intentional data quality issues are injected into the raw data to simulate real-world problems — approximately 3% null values, 2% duplicates, 1% invalid store IDs, and a small percentage of future-dated transactions and negative quantities.

The Architecture

The pipeline follows the **Medallion Architecture** — an industry standard pattern for organising data in three quality layers:



The Technology Stack		
Tool	Role	Why It Was Chosen
Python + Faker	Data generation	Industry standard for synthetic data; simulates real-world dirty data

Tool	Role	Why It Was Chosen
Databricks Free Edition	Bronze + Silver processing	Enterprise-grade Spark platform; same tool used in production at major companies
PySpark	Data transformation in Databricks	Handles large-scale data processing; standard in data engineering roles
Delta Lake	Storage format	ACID transactions, time travel, schema enforcement — production standard
dbt Core	Gold layer transformation	SQL-based transformations with built-in testing, documentation, and lineage
Unity Catalog	Data governance	Manages catalogs, schemas, and access control in Databricks
Python-dotenv	Secret management	Keeps credentials out of code — same pattern as Azure Key Vault
Total monthly cost: \$0		

The Six Gold Models

These are the final analytical tables produced by the pipeline. Each one answers specific business questions.

1. gold_sales_daily

Grain: One row per (date, store, product, payment method)

Business question: How much did each store sell of each product each day, and how were customers paying?

Key metrics: Total transactions, total units sold, total revenue, average basket value, gross revenue, discount given, unique customers

2. gold_store_performance

Grain: One row per (store, date)

Business question: How is each store performing day by day? Which regions are strongest?

Key metrics: Total transactions, total sales, average basket value, unique customers, units sold, discounts given, distinct products sold

3. gold_product_performance

Grain: One row per product

Business question: Which products are generating the most revenue? What is the gross margin on each product?

Key metrics: Units sold, total revenue, average selling price, average discount percentage, stores selling the product, gross margin percentage

4. gold_customer_lifetime_value

Grain: One row per customer

Business question: Who are our most valuable customers? How do we segment them by spending behaviour?

Key metrics: Total orders, total spend, average order value, first and last purchase date, customer tenure, purchase frequency band (VIP / Loyal / Returning / One-Time), spend band (High / Mid / Low Value)

5. gold_inventory_risk_daily

Grain: One row per (store, product)

Business question: Which products are at risk of running out of stock? Where should we prioritise restocking?

Key metrics: Stock on hand, reorder point, average daily units sold, days of stock remaining, stockout risk level (OK / Low / Critical / Out of Stock)

6. gold_promotion_effectiveness

Grain: One row per promotion

Business question: Are our promotions driving sales? How much revenue are they generating versus how much discount cost are they incurring?

Key metrics: Transactions with discount, units sold under promotion, discounted revenue, average discount applied, total discount cost, revenue per transaction

Data Quality Framework

One of the core features of the pipeline is its automated data quality system operating at two levels:

Silver Cleaning (Databricks)

Every entity has specific cleaning rules applied before data moves to Silver:

Entity	Rules Applied
Stores	Deduplicate on store_id, reject null region
Products	Reject null product_id, reject unit_price ≤ 0
Customers	Deduplicate on customer_id, reject null last_name
Transaction Headers	Reject null transaction_id, invalid store_id, future dates
Transaction Lines	Reject null IDs, quantity ≤ 0, unit_price ≤ 0
Inventory	Reject null IDs, invalid foreign keys, negative stock
Promotions	Reject null ID, discount ≤ 0, end_date before start_date

All rejected records are written to `smartretail.audit.rejected_records` with the rejection reason, entity name, and timestamp — creating a full audit trail.

Gold Testing (dbt)

53 automated tests run on every `dbt build`:

- **Not null** — critical columns are never empty
- **Unique** — primary keys have no duplicates
- **Accepted values** — categorical columns only contain expected values
- **Expression is true** — numerical columns meet business rules (e.g. total_sales ≥ 0)

If any test fails, dbt reports exactly which model and column has the issue before the data reaches any downstream consumer.

Daily Pipeline Run

The pipeline runs in 5 steps:

- Step 1 Generate data locally
python notebooks/00_generate_dirty_pos_data.py
- Step 2 Upload Parquet files to Databricks Volume
(via Databricks CLI or manual upload)
- Step 3 Run Bronze ingestion notebook in Databricks
Reads Parquet → writes smartretail.bronze.*
- Step 4 Run Silver cleaning notebook in Databricks
Cleans bronze.* → writes smartretail.silver.*
Rejections → smartretail.audit.rejected_records
- Step 5 Run dbt from local terminal
dbt build
Builds 6 Gold tables + runs 53 quality tests

Skills Demonstrated

This project demonstrates the following data engineering competencies:

Data Pipeline Design — end-to-end pipeline architecture following industry-standard medallion pattern with clear separation of concerns between ingestion, cleaning, and analytical layers

PySpark — DataFrame operations, window functions for deduplication, union operations for rejection tracking, Delta Lake writes with schema merging

dbt — SQL model development, source and ref functions, Jinja templating, schema testing, documentation generation, lineage graphs, dbt-databricks adapter configuration

Data Quality Engineering — multi-layer quality framework with Silver cleaning rules and Gold automated testing; audit trail for rejected records

Delta Lake — ACID-compliant storage, append and overwrite modes, schema evolution with mergeSchema

Databricks — Unity Catalog setup, serverless compute, SQL Warehouse configuration, Volume storage, SQL Editor

Python — synthetic data generation, environment variable management with dotenv, Parquet file handling with pandas and pyarrow

Developer Practices — `.gitignore` for secret management, `.env.template` pattern for credential sharing, modular project structure

SmartRetail POS Intelligence Pipeline Stack: Python · PySpark · Databricks Free Edition · dbt Core · Delta Lake Cost: \$0/month